

Microprocessor and Computer Hardware

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Microprocessor and Computer Hardware — Complete Notes	4
1. Microprocessor Fundamentals	4
1.1 Introduction to Microprocessors	4
1.2 Bus Organisation	5
2. 8085 / 8080A Microprocessor	6
2.1 Architecture	6
2.2 Registers	6
2.3 Pin Diagram and Signals	7
3. 8085 Programming	9
3.1 Instruction Set and Addressing Modes	9
3.2 Stacks and Subroutines	9
3.3 Assembly Language Programming Basics	10
4. Interrupts	10
4.1 Interrupt Concepts and Classification	10
4.2 8085 Interrupt Structure	10
4.3 Programmable Interrupt Controller (8259)	11
5. Memory	12
5.1 Memory Classification	12
5.2 Memory Types (RAM / ROM)	12
6. Digital Electronics — Sequential Circuits	13
6.1 Flip-Flops	13
6.2 Counters	13
6.3 Registers and Shift Registers	14
7. Digital Electronics — Combinational Circuits	14
7.1 Multiplexer (MUX) and Demultiplexer (DEMUX)	14
7.2 Encoder and Decoder	15
7.3 Digital Comparator	15
7.4 Parity Generator / Checker	15
7.5 Adders and Subtractors	16
8. Data Converters	16
8.1 D/A Converters (Digital-to-Analog)	16
8.2 A/D Converters (Analog-to-Digital)	16
9. Computer Hardware Basics	17
9.1 Motherboard, Chipset, Ports and Connectors	17
9.2 Peripheral Interfacing	17
9.3 I/O Devices and Power Supply	17

10. Quick Revision — Exam Pointers	17
--	----

Microprocessor and Computer Hardware — Complete Notes

1. Microprocessor Fundamentals

1.1 Introduction to Microprocessors

A **microprocessor** is a single **VLSI** (Very Large Scale Integration) chip that contains the entire **CPU** (Central Processing Unit) — the **ALU**, the **control unit**, and the **register array** — fabricated on one piece of silicon. It is a **programmable, clock-driven, register-based, digital** electronic device that reads binary instructions from memory, accepts binary data as input, and processes that data according to the instructions.

The microprocessor is often called the **brain** of the computer. The very first microprocessor was the **Intel 4004**, released in **1971**, which was a **4-bit** device.

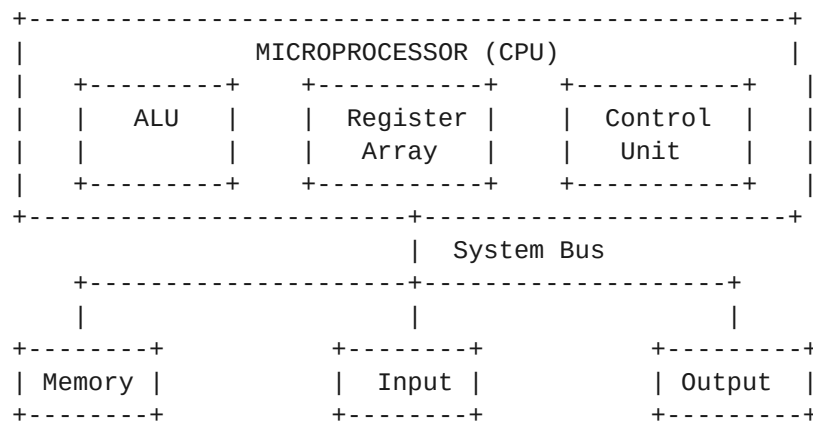
1.1.1 Microprocessor Architecture and Its Operations

Every microprocessor performs three fundamental operations that together form the **fetch-decode-execute cycle**:

- **Fetch** — read the instruction (opcode) from memory pointed to by the Program Counter.
- **Decode** — the control unit interprets the opcode to determine the required action.
- **Execute** — perform the operation (arithmetic, logic, data transfer, or branch).

The microprocessor performs four basic functional operations:

- **Memory read / Memory write** — transfer of data between the CPU and memory.
- **I/O read / I/O write** — transfer of data between the CPU and input/output devices.
- **Arithmetic and Logic operations** — performed by the **ALU**.
- **Decision making and branching** — altering program flow based on flag conditions.



1.1.2 Evolution and Generations of Microprocessors

Generation	Year	Word Size	Example	Note
First	1971	4-bit	Intel 4004	First microprocessor ever
Second	1974–76	8-bit	Intel 8008, 8080, 8085	NMOS technology
Third	1978	16-bit	Intel 8086 / 8088	Segmented memory
Fourth	1980s	32-bit	Intel 80386	On-chip MMU
Fifth	1993+	64-bit	Pentium and later	Superscalar, pipelining

1.1.3 Microprocessor vs Microcontroller

Feature	Microprocessor	Microcontroller
Contents	Only CPU on chip	CPU + RAM + ROM + I/O + Timer on one chip
Memory/I/O	External	Internal (on-chip)
Architecture	Usually Von Neumann	Usually Harvard
Purpose	General purpose	Specific/embedded purpose
Cost / Power	Higher	Lower
Example	8085, 8086	8051, PIC, AVR

MCQ Trap: A microcontroller is a **complete computer on a chip** (CPU + memory + I/O). A microprocessor is **only the CPU** and needs external memory and I/O. Examiners frequently swap these definitions.

1.1.4 Word Size, Clock Speed and Instruction Set

- **Word size / Word length** — number of bits the CPU processes at a time; the **8085 is an 8-bit** processor (8-bit word).
- **Clock speed** — number of cycles per second, measured in **Hz**; the **8085A** operates at a maximum of **3 MHz**.
- **Instruction set** — the complete collection of commands a processor understands; the **8085** has **74** instruction types and **246** opcodes.

1.2 Bus Organisation

A **bus** is a group of conducting wires that carry information (bits) in parallel between the microprocessor and other components. The **system bus** is collectively made of three buses.

1.2.1 The Three System Buses

Bus	Width in 8085	Direction	Carries
Address Bus	16-bit (A0–A15)	Unidirectional	Memory / I/O addresses
Data Bus	8-bit (D0–D7)	Bidirectional	Data and instructions
Control Bus	Several lines	Mixed	Control signals (RD, WR, ALE, IO/M)

- The **16-bit address bus** lets the 8085 address $2^{16} = 65,536 = 64 \text{ KB}$ of memory.
- The **8-bit data bus** means the largest data unit transferred in one operation is **1 byte**.
- For I/O-mapped I/O the 8085 uses an **8-bit** I/O address → **256** input ports and **256** output ports.

```

ADDRESS BUS (16-bit, unidirectional) ---->
+-----+ =====+-----+
| 8085 | <===== DATA BUS (8-bit, bidirectional) | MEMORY |
| CPU | ===== | & I/O |
+-----+ <----- CONTROL BUS (RD,WR,ALE, IO/M) --> +-----+

```

Memory Trick: "Address is Always Away (unidirectional, outgoing); Data is Dual-direction (bidirectional)."

MCQ Trap: The address bus is **unidirectional**; the data bus is **bidirectional**. Memory capacity depends on **address-bus width** (2^n), NOT the data-bus width.

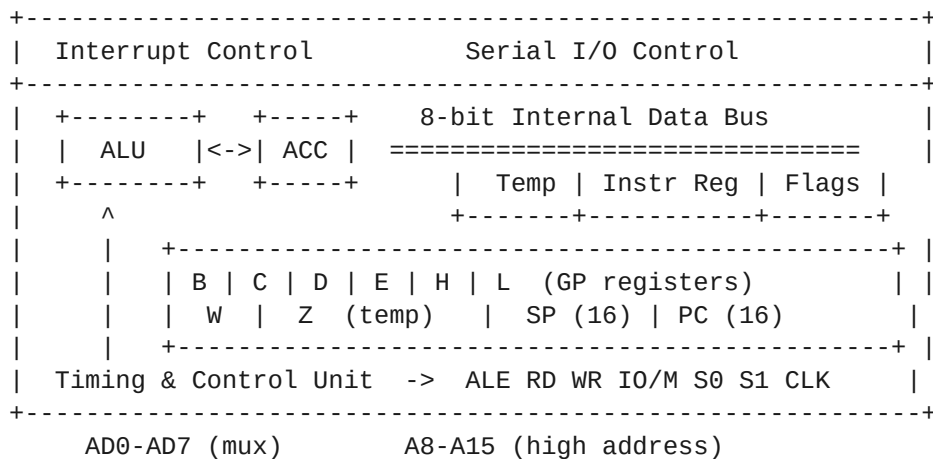
2. 8085 / 8080A Microprocessor

2.1 Architecture

The **Intel 8085** is an **8-bit** general-purpose microprocessor introduced in **1976**, built on **NMOS** technology with about **6,500 transistors**, housed in a **40-pin DIP** (Dual In-line Package), requiring a single **+5 V** supply.

2.1.1 8085 Internal Architecture (Functional Blocks)

- **ALU (Arithmetic Logic Unit)** — performs **8-bit** arithmetic (add, subtract, increment, decrement) and logic (AND, OR, XOR, complement, compare) operations; works with the **Accumulator** and **temporary register**, and sets the **flags**.
- **Timing and Control Unit** — generates control and timing signals (**ALE, RD, WR, IO/M**) to synchronise internal and external operations; it is the "nerve centre".
- **Instruction Register and Decoder** — the **IR** holds the fetched opcode; the decoder interprets it.
- **Register Array** — general-purpose and special-purpose registers.
- **Interrupt Control** — handles the five hardware interrupts.
- **Serial I/O Control** — handles serial data via **SID** and **SOD** pins.



2.1.2 8080A Architecture and Comparison with 8085

The **Intel 8080A** (released **1974**) is the predecessor of the 8085. The 8085 is **software-compatible** with the 8080A (the name "8085" denotes a single +5 V supply).

Feature	8080A	8085
Power supply	+5 V, -5 V, +12 V (three)	+5 V (single)
Clock generator	External (needs 8224)	On-chip (needs only crystal)
System controller	External (needs 8228)	On-chip
Serial I/O	Not available	SID / SOD pins
Hardware interrupts	1 (INT)	5 (TRAP, RST 7.5/6.5/5.5, INTR)
Extra instructions	—	SIM and RIM
Clock frequency	2 MHz	3 MHz

MCQ Trap: The 8085 added exactly **two** new instructions over the 8080A — **SIM** (Set Interrupt Mask) and **RIM** (Read Interrupt Mask). All other 8080A instructions are retained.

2.2 Registers

The 8085 register set consists of six 8-bit general-purpose registers, one accumulator, a flag register, and two 16-bit special registers, plus internal temporary registers.

2.2.1 General-Purpose Registers (B, C, D, E, H, L)

- There are **6** general-purpose **8-bit** registers: **B, C, D, E, H, L**.
- They can be combined into three **16-bit register pairs**: **BC, DE, HL**.
- The **HL pair** is special: it acts as a **memory pointer** and is referenced as **M** (memory) in instructions like **MOV A, M**.

2.2.2 Accumulator (A)

- The **Accumulator (A)** is an **8-bit** register that is part of the ALU.
- It holds **one operand** before an operation and stores the **result** after the operation.
- Most arithmetic/logic instructions use the accumulator implicitly.

2.2.3 Program Counter (PC) and Stack Pointer (SP)

- **Program Counter (PC)** — a **16-bit** register that holds the **address of the next instruction** to be fetched; it is auto-incremented after each fetch.
- **Stack Pointer (SP)** — a **16-bit** register that holds the **address of the top of the stack**; the stack grows toward **lower** addresses.

2.2.4 Flag Register (S, Z, AC, P, CY)

The **8085 flag register** is **8-bit** wide but uses only **5 flags**; bits **D5, D3, D1** are **unused**.

```

Bit:  D7   D6   D5   D4   D3   D2   D1   D0
      +---+---+---+---+---+---+---+---+
      | S | Z | x | AC | x | P | x | CY |
      +---+---+---+---+---+---+---+---+

```

Flag	Bit	Name	Set when
S	D7	Sign	Result MSB = 1 (result negative)
Z	D6	Zero	Result = 00H (zero)
AC	D4	Auxiliary Carry	Carry out of bit 3 (used by DAA)
P	D2	Parity	Result has even number of 1s
CY	D0	Carry	Carry/borrow out of bit 7

Memory Trick: Flag order high-to-low — "**S Z (x) AC (x) P (x) CY**". Mnemonic: "**Some Zebras Are Purely Crazy**."

MCQ Trap: **AC** (Auxiliary Carry) cannot be tested by conditional jump instructions; it is used **only** internally by the **DAA** instruction for BCD arithmetic.

2.2.5 Instruction Register and Temporary Registers

- The **Instruction Register (IR)** stores the opcode currently being decoded; it is **not** accessible to the programmer.
- **W** and **Z** are **8-bit** internal temporary registers used by the processor during instruction execution; they are also **not** programmer-accessible.

2.3 Pin Diagram and Signals

The 8085 is a **40-pin** IC. Its pins are grouped into address/data lines, control & status signals, power & clock, interrupts, serial I/O, and reset.

2.3.1 Address and Data Lines

- **A8–A15** (pins 21–28) — higher-order **address bus** (unidirectional).
- **AD0–AD7** (pins 12–19) — **multiplexed** lower-order address/data bus; carries lower address during the first clock state and data afterwards.

2.3.2 Control and Status Signals

Signal	Type	Active	Function
ALE	Control	High	Address Latch Enable ; high in T1 to latch lower address
RD	Control	Low	Read control for memory/I/O
WR	Control	Low	Write control for memory/I/O
IO/M	Status	—	High = I/O, Low = Memory operation
S0, S1	Status	—	Indicate the type of machine cycle
READY	Control	High	If low, CPU waits (for slow memory/I/O)
HOLD	Control	High	DMA request to release the bus
HLDA	Control	High	Hold Acknowledge — bus released

The combination of **IO/M, S1, S0** identifies the machine cycle:

IO/M	S1	S0	Machine Cycle
0	1	1	Opcode Fetch
0	1	0	Memory Read
0	0	1	Memory Write
1	1	0	I/O Read
1	0	1	I/O Write
1	1	1	Interrupt Acknowledge

2.3.3 Power Supply and Clock Frequency

- **Vcc** (pin 40) — **+5 V** single power supply.
- **Vss** (pin 20) — **ground** (0 V).
- **X1, X2** (pins 1, 2) — crystal/RC/LC inputs to the internal clock generator.
- **CLK OUT** (pin 37) — clock output for peripherals.
- The operating clock frequency is **half** the crystal frequency, so a **6 MHz** crystal gives **3 MHz** internal operation.

MCQ Trap: The internal clock = **crystal frequency ÷ 2**. For 3 MHz operation you connect a **6 MHz** crystal across X1–X2.

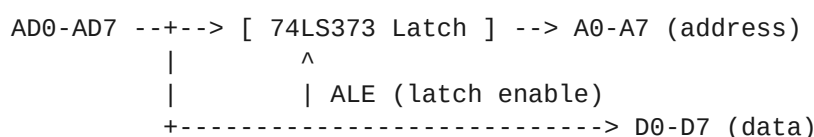
2.3.4 Interrupt, Serial and Reset Pins

- Interrupt inputs: **TRAP** (pin 6), **RST 7.5** (pin 7), **RST 6.5** (pin 8), **RST 5.5** (pin 9), **INTR** (pin 10); **INTA** (pin 11) is the acknowledge output.
- Serial pins: **SID** (pin 5, Serial Input Data) and **SOD** (pin 4, Serial Output Data).
- Reset pins: **RESET IN** (pin 36, active low) and **RESET OUT** (pin 3).

2.3.5 Demultiplexing the Address/Data Bus

Because **AD0–AD7** carry both the lower address and data at different times, they must be **demultiplexed** to separate the address:

- During **T1**, **ALE = 1** and AD0–AD7 hold the **lower address byte**.
- An external **latch (74LS373)** captures this address on the falling edge of **ALE**.
- For the rest of the machine cycle, **ALE = 0** and AD0–AD7 carry **data**.



3. 8085 Programming

3.1 Instruction Set and Addressing Modes

An **instruction** is a binary command that directs the microprocessor to perform a specific operation. Each instruction has two parts: the **opcode** (operation code — what to do) and the **operand** (the data or address — on what to do it).

3.1.1 Instruction Format (1, 2, 3 byte)

Type	Size	Contents	Example
1-byte	1 byte	Opcode only	MOV A, B, ADD B, CMA
2-byte	2 bytes	Opcode + 8-bit data/port	MVI A, 32H, IN 01H
3-byte	3 bytes	Opcode + 16-bit address/data	LDA 2050H, JMP 2000H, LXI H, 2050H

MCQ Trap: In a 3-byte instruction the **second byte is the lower-order** address/data and the **third byte is the higher-order** byte (little-endian storage).

3.1.2 Instruction Classification (5 Groups)

- **Data transfer (copy) instructions** — MOV, MVI, LDA, STA, LXI, LDAX, STAX, LHLD, SHLD, XCHG, IN, OUT. These do **not** affect the flags.
- **Arithmetic instructions** — ADD, ADI, SUB, SUI, INR, DCR, INX, DCX, DAD, DAA. These affect the flags (except INX/DCX).
- **Logical instructions** — ANA, ANI, ORA, ORI, XRA, XRI, CMP, CPI, RLC, RRC, RAL, RAR, CMA, CMC, STC.
- **Branching instructions** — JMP, conditional jumps, CALL, RET, RST, PCHL.
- **Machine/Control instructions** — NOP, HLT, DI, EI, SIM, RIM.

MCQ Trap: Data transfer instructions only **copy** (the source is unchanged) and **do NOT modify flags**. Many candidates wrongly assume MOV/MVI change the Zero flag.

3.1.3 Addressing Modes of 8085 (5 Modes)

Mode	Description	Example
Immediate	Operand (data) given in instruction	MVI A, 05H, LXI H, 2000H
Register	Operands are in registers	MOV A, B, ADD C
Direct	16-bit memory address in instruction	LDA 2000H, STA 2000H
Register Indirect	Address held in a register pair	MOV A, M, LDAX B
Implied / Implicit	No operand specified explicitly	CMA, RAL, STC, CMC

Memory Trick: "I R D I I" → Immediate, Register, Direct, register Indirect, Implied — the **5** addressing modes of 8085.

3.2 Stacks and Subroutines

3.2.1 Stack and PUSH / POP

- The **stack** is a reserved area of **R/W memory** used for temporary storage, operating on a **LIFO** (Last In, First Out) basis.
- The **Stack Pointer (SP)** always points to the **top of the stack**.
- In the 8085 the stack grows **downward** (toward lower addresses).
- **PUSH rp** — stores a register pair on the stack: SP is **decremented by 2**; the **high byte** is stored first at (SP–1), then the **low byte** at (SP–2).

- **POP rp** — retrieves a register pair: the **low byte** is read from (SP), the **high byte** from (SP+1); SP is **incremented by 2**.
- Stack operations act on register pairs **BC, DE, HL, PSW** (Processor Status Word = **A + flags**).

PUSH B (SP=2050H)	POP B
2050: (top before)	reads (SP), (SP+1)
204F: <- B (high)	SP increases by 2
204E: <- C (low) SP=204E	(LIFO order)

MCQ Trap: **PUSH decrements** SP first; **POP increments** SP after. The high (first) register of the pair is stored at the **higher** address.

3.2.2 Subroutines (CALL and RET)

- A **subroutine** is a reusable group of instructions written once and called many times.
- **CALL addr** — a **3-byte** instruction that **pushes the return address** (current PC) onto the stack and jumps to the subroutine. It takes **18 T-states**.
- **RET** — pops the return address from the stack back into the PC, returning to the main program. It takes **10 T-states**.
- Conditional CALL/RET execute based on flag status.

3.2.3 Nesting of Subroutines

- **Nesting** means one subroutine calls another subroutine.
- The **stack** automatically preserves multiple return addresses in LIFO order, so the program returns correctly through each level.

3.3 Assembly Language Programming Basics

- **Machine language** uses binary opcodes; **assembly language** uses human-readable **mnemonics** (e.g., **MOV, ADD**).
- An **assembler** translates assembly mnemonics into machine code.
- **Timing terms:** a **T-state** is one clock period; a **machine cycle** is the time to do one memory/I/O access; an **instruction cycle** is fetch + execute of one instruction.
- An opcode-fetch machine cycle takes **4 T-states** (up to 6 for some); 8085 instructions take **1 to 5** machine cycles and **4 to 18** T-states.

4. Interrupts

4.1 Interrupt Concepts and Classification

An **interrupt** is a signal that temporarily suspends the normal program execution so the CPU can service a higher-priority event; afterward control returns to the interrupted program. This avoids inefficient **polling**.

When an interrupt is accepted, the CPU completes the current instruction, saves the **PC** on the **stack**, and jumps to the corresponding **Interrupt Service Routine (ISR)** at a fixed **vector address**.

Classification	Categories
By source	Hardware (via pins) vs Software (via instructions RST 0–7)
By masking	Maskable (can be disabled) vs Non-maskable (cannot)
By vector	Vectored (fixed address) vs Non-vectored (address supplied by device)

4.2 8085 Interrupt Structure

The 8085 has **5 hardware interrupts**. Their priority from highest to lowest is **TRAP > RST 7.5 > RST 6.5 > RST 5.5 > INTR**.

Interrupt	Type	Maskable?	Vector Address	Trigger
TRAP	Hardware	No (non-maskable)	0024H	Edge + Level
RST 7.5	Hardware	Yes	003CH	Rising edge
RST 6.5	Hardware	Yes	0034H	Level
RST 5.5	Hardware	Yes	002CH	Level
INTR	Hardware	Yes	None (non-vectorized)	Level

- **Vector address formula:** for RST *n*, vector address = $*n* \times 8$. Example: RST 5.5 $\rightarrow 5.5 \times 8 = 44 = 002CH$.
- **TRAP** is also known as **RST 4.5** ($4.5 \times 8 = 36 = 0024H$) and has the **highest** priority; it **cannot be disabled**, making it ideal for power-failure / emergency handling.
- **INTR** is the **lowest-priority, non-vectorized** interrupt — the interrupting device must supply the instruction (typically an RST or CALL) during the **INTA** cycle.

Software interrupts **RST 0–RST 7** are **1-byte CALL** instructions to fixed addresses:

Instruction	Vector	Instruction	Vector
RST 0	0000H	RST 4	0020H
RST 1	0008H	RST 5	0028H
RST 2	0010H	RST 6	0030H
RST 3	0018H	RST 7	0038H

Interrupt-control instructions:

- **EI** — Enable Interrupts; **DI** — Disable Interrupts.
- **SIM** — Set Interrupt Mask (masks RST 7.5/6.5/5.5 and drives SOD).
- **RIM** — Read Interrupt Mask (reads mask status, pending interrupts, and SID).

MCQ Trap: **TRAP** is the only **non-maskable** 8085 interrupt and is **edge-and-level** triggered so a noise spike alone cannot trigger it. **INTR** is the only **non-vectorized** one.

Memory Trick: Vector addresses of TRAP, 5.5, 6.5, 7.5 = **24, 2C, 34, 3C** (H) — they increase by **8** each step.

4.3 Programmable Interrupt Controller (8259)

The **Intel 8259** is a **Programmable Interrupt Controller (PIC)** used to expand and manage interrupts.

- A single 8259 manages **8** interrupt request lines: **IR0–IR7**.
- 8259s can be **cascaded**: **1 master + 8 slaves** handle up to **64** interrupts.
- In fully-nested mode **IR0** has the **highest** priority and **IR7** the **lowest**.
- It is programmed by **ICW1–ICW4** (Initialization Command Words) and **OCW1–OCW3** (Operation Command Words).
- Operating modes include **fully nested**, **automatic/specific rotation**, **special mask**, and **polled** mode.

MCQ Trap: One 8259 handles **8** interrupts; a cascade of **1 master + 8 slaves** handles **64** (not 72) — the master's lines are used to connect slaves.

5. Memory

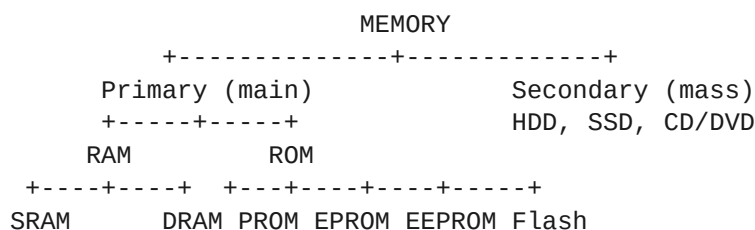
5.1 Memory Classification

5.1.1 Primary vs Secondary Memory

Aspect	Primary Memory	Secondary Memory
Access by CPU	Direct	Indirect (via I/O)
Speed	Fast	Slow
Cost per bit	High	Low
Volatility	Mostly volatile	Non-volatile
Examples	RAM, ROM, Cache	HDD, SSD, CD, DVD, Pen drive

5.1.2 Volatile vs Non-volatile

- **Volatile memory** loses its contents when power is removed — e.g., **RAM (SRAM, DRAM)**.
- **Non-volatile memory** retains contents without power — e.g., **ROM, Flash, HDD, SSD**.



5.2 Memory Types (RAM / ROM)

5.2.1 R/W Memory — RAM (SRAM vs DRAM)

RAM (Random Access Memory) is **read/write**, **volatile** main memory.

Feature	SRAM	DRAM
Storage cell	Flip-flop (6 transistors)	1 transistor + 1 capacitor
Refresh needed	No	Yes (periodic)
Speed	Faster	Slower
Density / capacity	Lower	Higher
Cost	Expensive	Cheaper
Typical use	Cache memory	Main memory

MCQ Trap: **SRAM** uses **flip-flops** and needs **no refresh**; **DRAM** uses a **capacitor** and **must be refreshed** because the charge leaks. Examiners often reverse "which one needs refresh".

5.2.2 ROM (PROM, EPROM, EEPROM, Flash)

ROM (Read Only Memory) is **non-volatile** memory used to store permanent programs like the **BIOS / monitor**.

Type	Programmed by	Erased by	Reusable?
Mask ROM	Manufacturer	Cannot be erased	No
PROM	User (once)	Cannot be erased	One-time (OTP)
EPROM	User	UV light	Yes (full chip)
EEPROM	User	Electrically	Yes (byte-wise)
Flash	User	Electrically	Yes (block/sector)

MCQ Trap: EPROM is erased by **ultraviolet (UV) light** through a quartz window; EEPROM is erased **electrically**. Flash is a fast, block-erasable form of EEPROM used in **USB drives and SSDs**.

5.2.3 RAM Organization and Memory Interfacing with 8085

- A memory chip of capacity $2^n \times m$ has **n address lines** and **m data lines**; e.g., a **1K × 8** chip has **10 address lines** ($2^{10} = 1024$) and **8 data lines**.
- Memory mapping** assigns address ranges to chips; **address decoding** (using decoders like **74138**, or NAND gates) selects the correct chip via its **Chip Select (CS)**.
- The 8085 uses **IO/M** to distinguish memory (**IO/M = 0**) from I/O (**IO/M = 1**) accesses.

Shortcut: Address lines = **$\log_2(\text{number of locations})$** . A 64 KB memory needs **16** address lines because $2^{16} = 65,536$.

6. Digital Electronics — Sequential Circuits

A **sequential circuit's** output depends on the **present inputs AND past state** (it has memory), unlike a combinational circuit. The basic memory element is the **flip-flop**.

6.1 Flip-Flops

6.1.1 Latch vs Flip-Flop

Aspect	Latch	Flip-Flop
Triggering	Level -triggered	Edge -triggered
Clock	Enable-based / no clock	Clock -controlled
Speed/Use	Simpler	Used in registers, counters

6.1.2 SR, JK, D, T Flip-Flops

Flip-Flop	Inputs	Special behaviour	Characteristic equation
SR	S, R	S=R=1 is forbidden	$Q_{\text{next}} = S + R'Q$
JK	J, K	J=K=1 toggles (fixes SR)	$Q_{\text{next}} = JQ' + K'Q$
D	D	Output follows input	$Q_{\text{next}} = D$
T	T	T=1 toggles , T=0 holds	$Q_{\text{next}} = T \oplus Q$

- SR (Set-Reset)** truth: S=0,R=0 → no change; S=0,R=1 → reset ($Q=0$); S=1,R=0 → set ($Q=1$); S=1,R=1 → **invalid**.
- JK** flip-flop removes the SR invalid state by **toggling** when J=K=1.
- D (Data/Delay)** flip-flop stores one bit and is used in **registers** and **latches**.
- T (Toggle)** flip-flop is used to build **counters**.

6.1.3 Master-Slave Flip-Flop and Race-Around Condition

- The **race-around condition** occurs in a level-triggered **JK** flip-flop when **J=K=1** and the clock pulse width is **greater** than the propagation delay, causing the output to **toggle multiple times** in one clock pulse.
- It is eliminated by using a **master-slave** configuration (two flip-flops in series clocked on opposite edges) or by **edge-triggering**.

MCQ Trap: The **race-around** problem belongs to the **JK** flip-flop (when J=K=1). The **D** flip-flop does **not** have an invalid state or race-around problem.

6.2 Counters

A **counter** is a sequential circuit that counts clock pulses and progresses through a sequence of states. A counter with **n flip-flops** has a maximum modulus of 2^n (counts 0 to $2^n - 1$).

6.2.1 Asynchronous (Ripple) Counter

- Flip-flops are **not** clocked simultaneously; the output of one stage clocks the next.
- The clock "ripples" through the stages, causing cumulative **propagation delay** → **slower**.

6.2.2 Synchronous Counter

- All** flip-flops receive the **same clock** simultaneously → **faster**, no ripple delay.

6.2.3 Up / Down Counter

- An **up counter** counts **0 → max**; a **down counter** counts **max → 0**; an **up/down counter** does both based on a control input.

6.2.4 Ring Counter and Johnson Counter

Counter	Feedback	States with n FFs	Modulus
Ring	Q of last → D of first	n	MOD-n
Johnson (twisted-ring)	Q' of last → D of first	2n	MOD-2n

- A **ring counter** circulates a single **1** (or 0); with **n** flip-flops it has **n** states.
- A **Johnson (switch-tail) counter** feeds the **complemented** output back; with **n** flip-flops it has **2n** states.

6.2.5 Modulo-N Counters

- A **mod-N** counter has **N** distinct states (0 to N-1).
- Number of flip-flops needed = smallest integer **n** such that $2^n \geq N$, i.e., $n = \lceil \log_2 N \rceil$.
- Example: a **decade (MOD-10)** counter needs **4** flip-flops ($2^4 = 16 \geq 10$).

Memory Trick: Ring = n states, Johnson = 2n states for n flip-flops — "Johnson doubles".

MCQ Trap: A binary counter with **n** flip-flops counts up to $2^n - 1$ and has a maximum modulus of 2^n — off-by-one errors (2^n vs $2^n - 1$) are common exam traps.

6.3 Registers and Shift Registers

A **register** is a group of flip-flops storing multiple bits; an **n-bit** register uses **n flip-flops**. A **shift register** shifts data left or right on each clock.

Type	Input	Output
SISO	Serial	Serial
SIPO	Serial	Parallel
PISO	Parallel	Serial
PIPO	Parallel	Parallel

- SIPO** acts as a **serial-to-parallel** converter; **PISO** acts as a **parallel-to-serial** converter.
- An **n-bit SISO** register needs **n** clock pulses to shift data fully through.

7. Digital Electronics — Combinational Circuits

A **combinational circuit's** output depends **only** on the present inputs (no memory).

7.1 Multiplexer (MUX) and Demultiplexer (DEMUX)

7.1.1 Multiplexer (MUX) — Data Selector

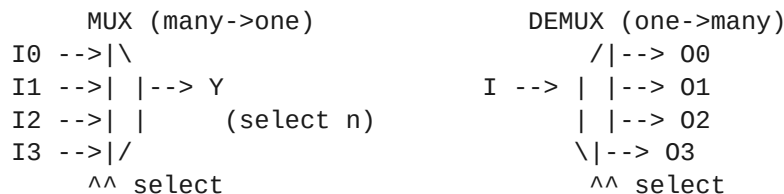
- A **MUX** is a **many-to-one** circuit: 2^n data inputs, **n** select lines, **1** output.
- It is also called a **data selector**.

MUX	Inputs	Select lines	Output
2:1	2	1	1

MUX	Inputs	Select lines	Output
4:1	4	2	1
8:1	8	3	1
16:1	16	4	1

7.1.2 Demultiplexer (DEMUX) — Data Distributor

- A **DEMUX** is a **one-to-many** circuit: **1** input, **n** select lines, **2^n** outputs.
- It is also called a **data distributor**.



MCQ Trap: A **MUX** has **many inputs / one output** (select = $\log_2(\text{inputs})$); a **DEMUX** has **one input / many outputs**. Number of select lines = $\log_2(\text{data lines})$.

7.2 Encoder and Decoder

- An **encoder** has **2^n inputs** and **n outputs** (many-to-few); e.g., an **8-to-3** (octal-to-binary) encoder. A **priority encoder** resolves multiple active inputs by priority.
- A **decoder** has **n inputs** and **2^n outputs** (few-to-many); e.g., a **3-to-8** (binary-to-octal) decoder, or a **BCD-to-7-segment** decoder.

Circuit	Inputs	Outputs
Encoder	2^n	n
Decoder	n	2^n

MCQ Trap: **Decoder** = n inputs $\rightarrow 2^n$ outputs (expands); **Encoder** = 2^n inputs $\rightarrow$ n outputs (compresses). They are functional opposites.

7.3 Digital Comparator

- A **digital comparator** compares two **n-bit** binary numbers A and B and produces three outputs: **$A > B$, $A = B$, $A < B$** .
- A **1-bit** comparator uses these expressions: equality **$(A=B) = A \oplus B$** ; **$(A>B) = A \cdot B'$** ; **$(A<B) = A' \cdot B$** .

7.4 Parity Generator / Checker

- A **parity bit** is an extra bit added to detect single-bit errors.
- Even parity:** the total number of 1s (data + parity) is **even**; **odd parity:** the total is **odd**.
- Parity generators/checkers are built from **XOR** (for even) and **XNOR** (for odd) gates.
- Parity detects **single-bit** (odd-number) errors but **cannot correct** them and misses **even-number** errors.

MCQ Trap: Parity can **detect** an odd number of bit errors but **cannot locate or correct** them, and it **fails** to detect an even number of errors.

7.5 Adders and Subtractors

Circuit	Inputs	Outputs	Sum/Diff	Carry/Borrow
Half Adder	2 (A, B)	2	$A \oplus B$	$A \cdot B$
Full Adder	3 (A, B, Cin)	2	$A \oplus B \oplus \text{Cin}$	$AB + \text{Cin}(A \oplus B)$
Half Subtractor	2 (A, B)	2	$A \oplus B$	$A' \cdot B$
Full Subtractor	3 (A, B, Bin)	2	$A \oplus B \oplus \text{Bin}$	$A'B + \text{Bin}(A \oplus B)'$

- A **half adder** adds two bits but has **no carry input**.
- A **full adder** adds three bits and can be built from **2 half adders + 1 OR gate**.
- An **n-bit ripple-carry adder** chains **n** full adders.

MCQ Trap: The **half adder** has **no carry-in**; the **full adder** has **three** inputs (A, B, Cin). Sum is always the **XOR** of the inputs.

8. Data Converters

Data converters bridge the digital and analog worlds. A **DAC** converts a digital code to an analog voltage/current; an **ADC** does the reverse.

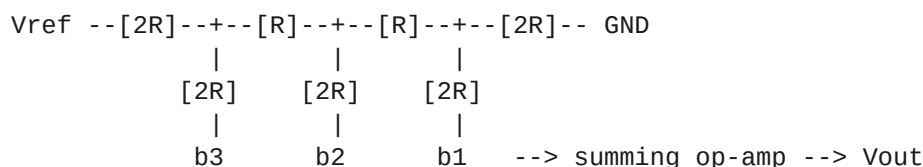
8.1 D/A Converters (Digital-to-Analog)

8.1.1 Binary-Weighted Resistor DAC

- Uses resistors with **binary-weighted** values (**R, 2R, 4R, 8R, ...**) fed into an **op-amp summing amplifier**.
- **Drawback:** for many bits it needs a very **wide range of precise resistor values**, which is hard to fabricate accurately.

8.1.2 R-2R Ladder Network DAC

- Uses only **two** resistor values — **R and 2R** — arranged in a ladder.
- **Advantages:** easy to fabricate (only two values), more **accurate**, and easily **scalable** to more bits → it is the **most widely used** DAC.



8.1.3 DAC Parameters

- **Resolution** — the smallest output step; an **n-bit** DAC has **2^n** output levels and step size = **$V_{ref} / (2^n - 1)$** .
- A higher number of bits → **finer** resolution.

MCQ Trap: The **R-2R ladder** uses only **two** resistor values (R and 2R); the **binary-weighted** DAC needs many different precise resistor values — that is its main disadvantage.

8.2 A/D Converters (Analog-to-Digital)

ADC Type	Speed	Notes
Flash (parallel)	Fastest	Needs $2^n - 1$ comparators for n bits; costly
Successive Approximation (SAR)	Medium	Binary search; n clock cycles for n bits; most common
Counter / Digital-ramp	Slow	Up to 2^n clock cycles
Dual-slope (integrating)	Slowest	High accuracy, excellent noise rejection

- **Flash ADC** is the **fastest** but most **expensive** because an **n-bit** flash needs **$2^n - 1$ comparators**.

- **SAR ADC** balances speed and cost using a **binary-search** algorithm taking **n** clock cycles.
- **Dual-slope ADC** is **slowest** but most **accurate**, used in **digital multimeters**.

8.2.1 Converter Performance Parameters

- **Resolution** — number of bits / smallest distinguishable step.
- **Accuracy** — how close the output is to the true value.
- **Conversion time** — time taken to complete one conversion (lowest for **flash**, highest for **dual-slope**).

Memory Trick: ADC speed ranking — "**Flash is fast, Dual-slope is slow**"; SAR sits in the middle.

9. Computer Hardware Basics

9.1 Motherboard, Chipset, Ports and Connectors

- The **motherboard** is the main **PCB** that interconnects the CPU, memory, expansion slots, and peripherals.
- The **chipset** manages data flow; traditionally split into the **Northbridge** (fast: CPU, RAM, graphics) and **Southbridge** (slower I/O: USB, SATA, audio).
- Common ports/connectors: **USB, HDMI, VGA, Ethernet (RJ45), SATA, PS/2, audio jacks**.
- Expansion slots include **PCI** and **PCIe** (PCI Express).

9.2 Peripheral Interfacing

9.2.1 8255 PPI (Programmable Peripheral Interface)

- The **8255** is a **40-pin** general-purpose programmable I/O chip.
- It has **3 ports** — **Port A, Port B, Port C** — each **8-bit**, giving **24** programmable I/O lines.
- **Port C** can be split into **upper (PC7–PC4)** and **lower (PC3–PC0)** nibbles.
- It supports **3 modes**: **Mode 0** (simple I/O), **Mode 1** (handshake I/O), **Mode 2** (bidirectional).

9.2.2 8253 / 8254 Programmable Interval Timer

- The **8253/8254** is a **Programmable Interval Timer (PIT)** with **3 independent 16-bit counters**.
- It supports **6 modes (Mode 0 to Mode 5)** such as interrupt-on-terminal-count and square-wave generation.
- The **8254** is the enhanced version of the **8253**, supporting **higher clock frequencies** (up to 10 MHz).

9.3 I/O Devices and Power Supply

- **Input devices:** keyboard, mouse, scanner, microphone. **Output devices:** monitor, printer, speaker.
- **SMPS (Switched-Mode Power Supply)** efficiently converts AC mains to regulated **DC** voltages (**+3.3 V, +5 V, +12 V**) for the PC; it is more **efficient** and **lighter** than a linear power supply.

Shortcut: Standard PC SMPS DC rails to remember: **+3.3 V, +5 V, +12 V** (and **–12 V**). The **+12 V** rail powers the CPU, fans, and drives.

10. Quick Revision — Exam Pointers

- **8085** is an **8-bit, 40-pin, +5 V, 3 MHz** NMOS microprocessor addressing **64 KB**.
- **Flag bits (D7→D0): S, Z, –, AC, –, P, –, CY**.
- **Interrupt priority:** **TRAP > RST 7.5 > RST 6.5 > RST 5.5 > INTR**; vectors **0024H, 003CH, 0034H, 002CH**.
- **Addressing modes (5):** Immediate, Register, Direct, Register-Indirect, Implied.
- **Ring counter = MOD-n; Johnson counter = MOD-2n** for n flip-flops.

- **MUX:** 2^n in $\rightarrow$ 1 out; **DEMUX:** 1 in $\rightarrow 2^n$ out; **Decoder:** $n \rightarrow 2^n$; **Encoder:** $2^n \rightarrow n$.
- **R-2R ladder DAC** uses only two resistor values; **Flash ADC** is fastest.
- **SRAM = flip-flops/no refresh; DRAM = capacitor/needs refresh.**
- **EPROM = UV-erased; EEPROM = electrically erased; Flash = block-erased EEPROM.**
- **8255** = 3 ports/24 lines/3 modes; **8259** = 8 IRs, cascade to 64; **8253/8254** = 3×16-bit counters/6 modes.